SQL query analysis and optimization for Postgres

Two branches of SQL optimization

There are two big branches of query optimization:

1. "Micro" optimization: analysis and improvement of particular queries. Main tools:
   - `EXPLAIN`
   - Database Lab that enables using `EXPLAIN` and DDL (to verify optimization ideas) on full-sized thin database clones.
2. "Macro" optimization: analysis of whole or large parts of workload, segmentation of it, studying characteristics, going from top to down, to identify and improve the parts that behave the worst. Main tools:
   - `pg_stat_statements` (for short, "pgss") for macro-analysis of SQL workload using Postgres-level metrics: execution and planning times, frequency, row volume, shared buffer reads and hits, WAL volume generated, and so on. Further here we'll use "pgss" to refer to pg_stat_statements.
   - `pg_stat_kcache` ("pgsk") to extend pgss analysis with physical-level metrics: user and system CPU time, context switches, real disk reads, writes, etc.
   - wait event analysis (a.k.a. Active Session History, ASH): pg_wait_sampling ("pgws") and extended Marginalia tooling.

Dashboards

Key dashboards to be used:

1. Postgres aggregated query performance analysis
2. Postgres single query performance analysis

Additional dashboards:

1. Postgres node performance overview (high-level) – this dashboard has many workload-related panels, for a high-level view at the whole workload, without segmentation by `queryid`
2. Postgres wait events analysis - this dashboard offers performance analysis centered around wait events, it provides same functionality as RDS Performance Insights.

How to use dashboards

All dashboards mentioned above deal with a single node. Thus, the first step is to make sure that the proper Postgres node is chosen:

- choose environment: grpd (default), gstg, or another;
- choose `type` (cluster name) – e.g., `patroni` is for gprd-main, `patroni-ci` is for gprd-ci;

- for the "Postgres aggregated query performance analysis" dashboard, choose TopN (default is 10) - this will define how many `queryid` lines will be visualized on panels for each metric;
- for the "Postgres single query performance analysis" dashboard, it is mandatory to specify `queryid` at the right top corner.

The "Postgres aggregated query performance analysis" dashboard provides "Top-N" charts for various metrics from pgss, pgsk, and pgws, as well as table view with lots. Both forms of information representation have their own pros and cons:

- each chart show only one metric, but also provides historical perspective (e.g., we can see if there was a spike and when),
- table view delivers consolidated view on aggregated queries (queryids), it has a lot of columns (almost every metric from pgss and pgsk is covered, with derivatives described below); these columns are sortable and provide a comprehensive understanding of the workload for certain period of time, but this data lack historical perspective.

The table view is collapsed by default – click on "pg_stat_statements and pg_stat_kcache views" to expand. Use these tables if you want to understand various characteristics of the workload. Don't forget that columns are sortable. For example, click on column `wal bytes %` if you, say, want what `queryid` contributed to WAL generation the most, and what was the percentage of that contribution.

pg_stat_statements basics

`pg_stat_statements` tracks all queries, aggregating them to query groups, called "normalized queries", where parameters are removed.

There are certain important limitations, some of which are worth remembering:

- it doesn't show anything about ongoing queries (can be found in `pg_stat_activity`);
- (a big issue!) it doesn't track failed queries, which can sometimes lead to wrong conclusions (example: CPU and disk IO load are high, but 99% of our queries fail on `statement_timeout`, loading our system but not producing any useful results – in this case, pgss is blind);
- if there are SQL comments, they are not removed, but only the first comment value is going to be present in the `query` column for each normalized query.

The view `pg_stat_statements` has 3 kinds of columns:

1. `queryid` – an identifier of normalized query. In the latest PG version it can also be used to connect (`JOIN`) data from pgss to pgsa (`pg_stat_statements`) and Postgres logs. Surprise: `queryid` value can be negative.

2. Descriptive columns: ID of database (`dbid`), user (`userid`), and the query itself (`query`).
3. Metrics. Almost all of them are cumulative: `calls`, `total_time`, `rows`, etc. Non-cumulative: `stddev_plan_time`, `stddev_exec_time`, `min_exec_time`, etc. In this post, we'll focus only on cumulative ones.

Let's mention some metrics that are usually most frequently used in macro optimization (full list):

1. `calls` – how many query calls happened for this query group (normalized query)
2. `total_plan_time` and `total_exec_time` – aggregated duration for planning and execution for this group (again, remember: failed queries are not tracked, including those that failed on `statement_timeout`)
3. `rows` – how many rows returned by queries in this group
4. `shared_blks_hit` and `shared_blks_read` – number if hit and read operations from the buffer pool. Two important notes here:
   - "read" here means a read from the buffer pool – it is not necessarily a physical read from disk, since data can be cached in the OS page cache. So we cannot say these reads are reads from disk.
   - The names "blocks hit" and "blocks read" might be a little bit misleading, suggesting that here we talk about data volumes – number of blocks (buffers). While aggregation here definitely make sense, we must keep in mind that the same buffers may be read or hit multiple times. So instead of "blocks have been hit" it is better to say "block hits".
5. `wal_bytes` – how many bytes are written to WAL by queries in this group

There are many more other interesting metrics, it is recommended to explore all of them (see the docs).

Dealing with cumulative metrics in pgss

To read and interpret data from pgss, you need three steps:

1. Take two snapshots corresponding to two points of time.
2. Calculate the diff for each cumulative metric and for time difference for the two points in time
   - a special case is when the first point in time is the beginning of stats collection – in PG14+, there is a separate view, `pg_stat_statements_info`, that has information about when the pgss stats reset happened; in PG13 and older this info is not stored, unfortunately.

3. (the most interesting part!) Calculate three types of derived metrics for each cumulative metric diff – assuming that M is our metric and remembering some basics of calculus from high school:
   a. `dM/dt` – time-based differentiation of the metric `M`;
   b. `dM/dc` – calls-based differentiation (I'll explain it in detail in the next post);
   c. `%M` – percentage that this normalized query takes in the whole workload considering metric `M`.

Step 3 here can be also applied not to particular normalized queries on a single host but bigger groups – for example:

- aggregated workload for all standby nodes
- whole workload on a node (e.g., the primary)
- bigger segments such as all queries from specific user or to specific database
- all queries of specific type – e.g., all `UPDATE` queries

Further we consider practical meanings of the three derivatives we discussed.

Derivative 1. Time-based differentiation

- `dM/dt`, where `M` is `calls` – the meaning is simple. It's QPS (queries per second). If we talk about particular group (normalized query), it's that all queries in this group have. `10,000` is pretty large so, probably, you need to improve the client (app) behavior to reduce it, `10` is pretty small (of course, depending on situation). If we consider this derivative for whole node, it's our "global QPS".

- `dM/dt`, where `M` is `total_plan_time + total_exec_time` – this is the most interesting and key metric in query macro analysis targeted at resource consumption optimization (goal: reduce time spent by server to process queries). Interesting fact: it is measured in "seconds per second", meaning: how many seconds our server spends to process queries in this query group. *Very* rough (but illustrative) meaning: if we have `2 sec/sec` here, it means that we spend 2 seconds each second to process such queries – we definitely would like to have more than 2 vCPUs to do that. Although, this is a very rough meaning because pgss doesn't distinguish situations when query is waiting for some lock acquisition vs. performing some actual work in CPU (for that, we need to involve wait event analysis) – so there may be cases when the value here is high not having a significant effect on the CPU load.

- `dM/dt`, where `M` is `rows` – this is the "stream" of rows returned by queries in the group, per second. For example, `1000 rows/sec` means a noticeable "stream" from Postgres server to client. Interesting fact here is that sometimes, we might need to think how much load the results produced by our Postgres server put on the application nodes – returning too many rows may require significant resources on the client side.

- `dM/dt`, where `M` is `shared_blks_hit + shared_blks_read` - buffer operations per second (only to read data, not to write it). This is another key metric for optimization. It is worth converting buffer operation numbers to bytes. In most cases, buffer size is 8 KiB (check: show block_size;), so `500,000` buffer hits&reads per second translates to `500000 bytes/sec * 8 / 1024 / 1024 = ~ 3.8 GiB/s` of the internal data reading flow (again: the same buffer in the pool can be process multiple times). This is a significant load – you might want to check the other metrics to understand if it is reasonable to have or it is a candidate for optimization.
- `dM/dt`, where `M` is `wal_bytes` – the stream of WAL bytes written. This is relatively new metric (PG13+) and can be used to understand which queries contribute to WAL writes the most – of course, the more WAL is written, the higher pressure to physical and logical replication, and to the backup systems we have. An example of highly pathological workload here is: a series of transactions like `begin; delete from ...; rollback;` deleting many rows and reverting this action – this produces a lot of WAL not performing any useful work. (Note: that despite the `ROLLBACK` here and inability of pgss to tracks failed statements, the statements here are going to be tracked because they are successful inside the transaction.)

Derivative 2. Calls-based differentiation

This set of metrics is not less important than time-based differentiation because it can provide you systematic view on characteristics of your workload and be a good tool for macro-optimization of query performance.

The metrics in this set help us understand the characteristics of a query, *on average*, for each query group.

Unfortunately, many monitoring systems disregard this kind of derived metrics. A good system has to present all or at least most of them, showing graphs how these values change over time (`dM/dc` time series).

Obtaining results for derived metrics of this kind is pretty straightforward:

- calculate difference of values `M` (the metric being studied) between two pgss snapshots: `M2 - M1`
- then, instead of using timestamps, get the difference of the "calls" values: `c2 - c1`
- then get `(M2 - M1) / (c2 - c1)`

Let's consider the meanings of various derived metrics obtained in such way:

1. `dM/dc`, where `M is calls` – a degenerate case, the value is always 1 (number of calls divided by the same number of calls).
2. `dM/dc`, where `M` is `total_plan_time + total_exec_time` – average query duration time in particular pgss group, a critically important metric for query performance observability. It can also be called "query

latency". When applied to the aggregated value for all normalized queries in pgss, its meaning is "average query latency on the server" (with two important comments that pgss doesn't track failing queries and sometimes can have skewed data due to the `pg_stat_statements.max` limit). The main cumulative statistics system in Postgres doesn't provide this kind of information – `pg_stat_database` tracks some time metrics, `blk_read_time` and `blk_write_time` if `track_io_timing` is enabled, and, in PG14+, `active_time` – but it doesn't have information about the number of statements (!), only the number for transactions, `xact_commit` & `xact_rollback`, is present; in some cases, we can obtain this data from other sources – e.g., pgbench reports it if we use it for benchmarks, and pgBouncer reports stats for both transaction and query average latencies, but in general case, in observability tools, pgss can be considered as the most generic way get the query latency information. The importance of it is hard to overestimate – for example:

- If we know that normally the avg query duration is <1 ms, then any spike to 10ms should be considered as a serious incident (if it happened after a deployment, this deployment should be reconsidered/reverted). For troubleshooting, it also helps to apply segmentation and determine which particular query groups contributed to this latency spike – was it all of them or just particular ones?
- In many cases, this can be taken as the most important metric for large load testing, benchmarks (for example: comparing average query duration for PG 15 vs. PG 16 when preparing for a major upgrade to PG 16).

3. `dM/dc`, where `M`is `rows` – average number of rows returned by a query in a given query group. For OLTP cases, the groups having large values (starting at a few hundreds or more, depending on the case) should be reviewed:
   - if it's intentional (say, data dumps), no action needed,
   - if it's a user-facing query and it's not related to data exports, then probably there is a mistake such as lack of `LIMIT` and proper pagination applied, then such queries should be fixed.
4. `dM/dc`, where `M` is `shared_blks_hit + shared_blks_read` – average number of "hits + reads" from the buffer pool. It is worth translating this to bytes: for example, `500,000` buffer hits&reads translates to `500000 GiB * 8 / 1024 / 1024 = ~ 3.8 GiB`, this is a significant number for a single query, especially if its goal is to return just a row or a few. Large numbers here should be considered as a strong call for query optimization. Additional notes:
   - in many cases, it makes sense to have hits and reads can be also considered separately – there may be the cases when, for example, queries in some pgss group do not lead to high disk IO and reading

from the page cache, but they have so many hits in the buffer pool, so their performance is suboptimal, even with all the data being cached in the buffer pool
- to have real disk IO numbers, it is worth using pg_stat_kcache
- a sudden change in the values of this metric for a particular group that persists over time, can be a sign of plan flip and needs to be studied
- high-level aggregated values are also interesting to observe, answering questions like "how many MiB do all queries, on average, read on this server?"

5. `dM/dc`, where `M` is `wal_bytes` (PG13+) – average amount of WAL generated by a query in the studied pgss group measured in bytes. It is helpful for identification of query groups that contribute most to WAL generation. A "global" aggregated value for all pgss records represents the average number of bytes for all statements on the server. Having graphs for this and for "`dM/dc`, where `M` is `wal_fpi`" can be very helpful in certain situations such as checkpoint tuning: with `full_page_writes = on`, increasing the distance between checkpoints, we should observe reduction of values in this area, and it may be interesting to study different particular groups in pgss separately.

3rd type of derived metrics: percentage

The third type of derived metrics is the percentage that a considered query group (normalized query or bigger groups such as "all statements from particular user" or "all `UPDATE` statements") takes in the whole workload with respect to metric `M`.

How to calculate it: first, apply time-based differentiation to all considered groups (as discussed in the part 1) — `dM/dt` — and then divide the value for particular group by the sum of values for all groups.

On the Postgres aggregated query performance analysis dashboard, this derivative is present in the table view (at the top of the dashboard), but can also be visually evaluated using left column of panels, because "stacked" option is being used for visualization.